BMB306 YAZILIM MÜHENDİSLİĞİ PROJE ANALİZ DOKÜMANI

1. PROJE TANIMI VE TAKIM YAPISI

Proje Adı: Çevrimiçi Sağlık Takip ve Randevu Sistemi

Kurum: Tekirdağ Namık Kemal Üniversitesi - Bilgisayar Mühendisliği Bölümü

1.1 Proje Tanımı ve Kapsamı

"Çevrimiçi Sağlık Takip ve Randevu Sistemi", hastane süreçlerini ve hasta-doktor etkileşimini dijitalleştirmeyi amaçlayan kapsamlı bir yazılım projesidir. Modern yazılım mühendisliği prensiplerine uygun olarak RESTful API mimarisiyle tasarlanan sistem, frontend ve backend katmanlarının bağımsız çalıştığı, ilişkisel bir veri tabanı (PostgreSQL) üzerine kuruludur. Proje, veri güvenliği ve yetkilendirme standartları çerçevesinde üç temel kullanıcı rolü sunar: Sistemin genel işleyişini, performans raporlamalarını ve departmanları yöneten Üst Düzey Yönetici (Admin); kendi randevu takvimini kontrol edip muayene sonrası dijital reçete yazabilen Orta Düzey Yönetici (Doktor); ve doktor/poliklinik bazlı filtreleme yaparak randevu alan, geçmiş sağlık verilerini takip eden Normal Kullanıcı (Hasta). Bu yapısıyla sistem, randevu karmaşasını ve fiziksel evrak yükünü ortadan kaldırarak kullanıcılara hızlı, güvenli ve izlenebilir bir dijital sağlık deneyimi sunmayı hedeflemektedir.

1.2 Proje Takımı ve Görev Dağılımı

Sistem, modern yazılım geliştirme prensiplerine uygun olarak rollerin ayrıştırıldığı bir ekip tarafından geliştirilecektir:

- **Muhammet Emin Erdem (Backend Geliştirici):** Sistem mimarisinin kurulması, Java (Spring Boot) ile RESTful API'lerin geliştirilmesi ve iş mantığının (business logic) kodlanması.
- **Ahmet Kerem Kerimoğlu (Frontend Geliştirici):** React.js kullanılarak kullanıcı arayüzlerinin geliştirilmesi, sayfa navigasyon modelinin oluşturulması ve API entegrasyonu.
- **Ömer Bal (Veri Tabanı Yöneticisi):** PostgreSQL veri tabanı tasarımı, ER diyagramlarının oluşturulması, tablolar arası ilişkilerin kurulması ve veri tutarlılığının sağlanması.
- **Süleyman Ata Ergin (Analiz ve Raporlama):** Gereksinim analizlerinin dokümante edilmesi, UML diyagramlarının hazırlanması, kullanım kılavuzunun yazılması ve test süreçlerinin takibi.

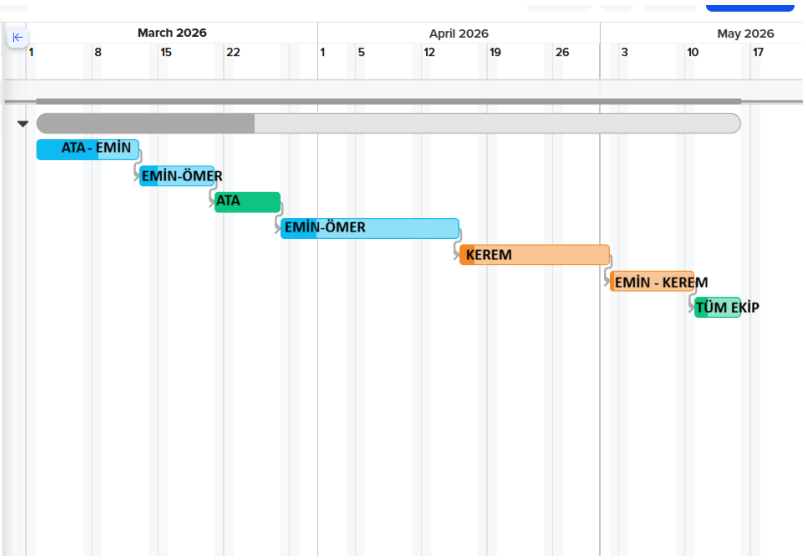
---

2. İŞ TANIMLARI VE AKTİVİTE LİSTESİ (ZAMAN PLANLAMASI)

Proje, belirtilen teslim tarihlerine uygun olarak şu aşamalardan oluşacaktır:

Tarih Aralığı	Aktivite	Sorumlu
02 - 12 Mart	Gereksinim Analizi ve Kullanım Senaryolarının (Use Case) Belirlenmesi	Ata - Emin
13 - 20 Mart	Sistem Mimarisinin Tasarımı (Class ve Sequence Diyagramları)	Emin - Ömer
21 - 27 Mart	Aşama Raporunun Tamamlanması ve Teslimi	Ata
28 Mart - 15 Nisan	Veri Tabanı Şemasının Oluşturulması ve API Taslaklarının Yazılması	Ömer - Emin
16 Nisan - 01 Mayıs	Arayüz Tasarımları ve Frontend Kodlamanın Başlaması	Kerem
02 - 10 Mayıs	Backend-Frontend Entegrasyonu ve Demo Testleri	Emin - Kerem
11 - 15 Mayıs	Final Teslimi: Canlı Demo ve Proje Raporu	Tüm Ekip

Name	Start date	End date	Progress
yazılım	2 Mar 26	15 May 26	31%
GÖREVLER	2 Mar 26	15 May 26	31%
Gereksinim Analizi ve Kullanım Senaryolarının (Use Case) Belirlenmesi	2 Mar 26	12 Mar 26	60%
Sistem Mimarisinin Tasarımı (Class ve Sequence Diyagramları)	13 Mar 26	20 Mar 26	25%
Aşama Raporunun Tamamlanması ve Teslimi	21 Mar 26	27 Mar 26	100%
Veritabanı Şemasının Oluşturulması ve API Taslaklarının Yazılması	28 Mar 26	15 Apr 26	20%
Arayüz Tasarımları ve Frontend Kodlamanın Başlaması	16 Apr 26	1 May 26	10%
Backend-Frontend Entegrasyonu ve Demo Testleri	2 May 26	10 May 26	5%
Final Teslimi: Canlı Demo ve Proje Raporu	11 May 26	15 May 26	29%



---

3. DETAYLI GEREKSİNİM ANALİZİ

3.1. Fonksiyonel Gereksinimler (Functional Requirements)

- **FR-1 (Kimlik Doğrulama):** Sistem; Hasta, Doktor ve Admin için güvenli giriş (JWT tabanlı) ve kaydolma özelliklerine sahip olmalıdır.
- **FR-2 (Randevu Yönetimi):** Hastalar branş ve doktora göre arama yapabilmeli, uygun saatlere randevu alabilmelidir.
- **FR-3 (Sağlık Geçmişi):** Doktorlar, muayene ettikleri hastaların geçmiş teşhislerini ve reçetelerini görüntüleyebilmelidir.
- **FR-4 (Reçete Sistemi):** Doktorlar dijital reçete oluşturabilmeli ve bu reçeteler hastanın panelinde listelenmelidir.
- **FR-5 (Raporlama):** Admin, aylık randevu sayılarını ve en çok tercih edilen branşları raporlayabilmelidir.

3.2. Fonksiyonel Olmayan Gereksinimler (Non-Functional Requirements)

- **NFR-1 (Güvenlik):** Kullanıcı şifreleri veri tabanında "salted hashing" (BCrypt) yöntemiyle şifreli tutulmalıdır.
- **NFR-2 (Performans):** Sistem, aynı anda 50 aktif kullanıcının işlem yapabileceği bir yükü desteklemelidir.
- **NFR-3 (Kullanılabilirlik):** Arayüzler, tıbbi terimlerin net anlaşıldığı sade ve kullanıcı dostu bir yapıda olmalıdır.

---

4. TEKNİK ALTYAPI (TECH STACK)

Verilen dokümanda önerilen modern teknolojiler tercih edilmiştir:

- **Backend:** Java 17+ ve Spring Boot (Spring Security, Spring Data JPA).
- **Frontend:** React.js (State yönetimi için Context API veya Redux).
- **Veritabanı:** PostgreSQL (İlişkisel veri tabanı zorunluluğu için).
- **Mimari:** RESTful API (Frontend ve Backend tamamen ayrık).

2. Kullanım Senaryoları (Use Case)

Aşağıdaki diyagram yapısını raporun "Use Case Diyagramı" kısmına eklemelisin.

1. HASTA SENARYOLARI (Normal Kullanıcı)

Senaryo 1: Randevu Alma (Hasta)

- **Ön Koşul:** Hastanın sisteme giriş yapmış olması.
- **Son Koşul:** Randevunun sisteme "Onaylandı" veya "Beklemede" olarak kaydedilmesi.
- **Ana Akış:**
    1. Hasta "Randevu Al" menüsüne tıklar.
    2. Sistem mevcut poliklinikleri listeler.
    3. Hasta poliklinik ve doktor seçer.
    4. Sistem doktorun boş takvimini (saatlerini) gösterir.
    5. Hasta uygun bir saat seçer ve onaylar.
- **Alternatif Akış:** * *4a:* Doktorun tüm saatleri doluysa sistem "Müsait saat bulunamadı" uyarısı verir ve hastayı başka bir doktora yönlendirir.

Senaryo 2: Reçete Görüntüleme

- **Ön Koşul:** Hastanın daha önce muayene olmuş ve adına reçete düzenlenmiş olması.
- **Akış:** Hasta "Reçetelerim" sayfasına girer; sistem tarih sırasına göre reçeteleri listeler, "Detay" butonuna basınca ilaç dozajı ve doktor notu görüntülenir.
- **Alternatif Akış:** Henüz hiç reçete yazılmamışsa "Kayıt bulunamadı" uyarısı verilir.

Senaryo 3: Profil Bilgilerini Güncelleme (Boy, Kilo vb.)

- **Ön Koşul:** Kullanıcının giriş yapmış olması.
- **Akış:** Hasta profil sayfasına girer; boy, kilo, kronik hastalıklar gibi alanları doldurur; "Güncelle"ye basar.
- **Son Koşul:** Veri tabanındaki Patient tablosu güncellenir ve doktorlar bu bilgileri randevu anında görür.

Senaryo 4: Doktor Değerlendirme ve Yorumlama (Yıldızlama)

- **Ön Koşul:** Randevunun "Tamamlandı" durumuna geçmiş olması.
- **Akış:** Hasta geçmiş randevularından ilgili doktoru seçer; 1-5 arası yıldız verir ve metin alanına yorum yazar.
- **Son Koşul:** Yorum admin onayına veya doğrudan doktor profiline düşer.

---

2. DOKTOR SENARYOLARI (Orta Düzey Yönetici)

Senaryo 5: Reçete Oluşturma (Doktor)

- **Ön Koşul:** Doktorun aktif bir randevu seansında olması.
- **Son Koşul:** Reçetenin veri tabanına kaydedilmesi ve hastaya bildirilmesi.
- **Ana Akış:**
    1. Doktor randevu listesinden hastayı seçer.
    2. "Reçete Yaz" butonuna tıklar.
    3. İlaç adı, dozaj ve notları girer.
    4. "Kaydet" butonuna tıklar.
- **Alternatif Akış:** * *3a:* İlaç adı sistemdeki ilaç listesinde yoksa doktor manuel giriş yapar veya uyarı alır.

Senaryo 6: Takvimden Hasta Bilgilerine Ulaşma

- **Ön Koşul:** Doktorun o gün onaylanmış randevularının olması.
- **Akış:** Doktor panelindeki takvimden ilgili hastanın ismine tıklar; sistem hastanın "Profil Güncelleme" senaryosunda girdiği boy, kilo ve geçmiş hastalık verilerini doktora sunar.
- **Alternatif Akış:** Hasta profilini doldurmamışsa sadece temel kimlik bilgileri görünür.

Senaryo 7: Randevu İptal Etme

- **Ön Koşul:** Acil bir durum oluşması veya randevu saatinin müsait olmaması.
- **Akış:** Doktor takvimden iptal edeceği randevuyu seçer; iptal nedenini girer; sistem hastaya bildirim gönderir.
- **Son Koşul:** Randevu tablosundaki status alanı CANCELLED_BY_DOCTOR olarak güncellenir.

---

3. ADMİN SENARYOLARI (Üst Düzey Yönetici)

Senaryo 8: Sistem İstatistikleri Raporlama (Admin)

- **Ön Koşul:** Kullanıcının Admin yetkisiyle giriş yapmış olması.
- **Son Koşul:** İstatistiki verilerin ekranda listelenmesi.

- **Ana Akış:**
    1. Admin "Raporlar" sekmesine girer.
    2. "Aylık Randevu İstatistikleri" filtresini seçer.
    3. Sistem veri tabanından aylık verileri çeker ve görselleştirir.
- **Alternatif Akış:**
    - *3a:* Seçilen tarih aralığında veri yoksa sistem "Kayıt bulunamadı" uyarısı döndürür.

Senaryo 9: Hastane Gelir-Gider Grafikleri

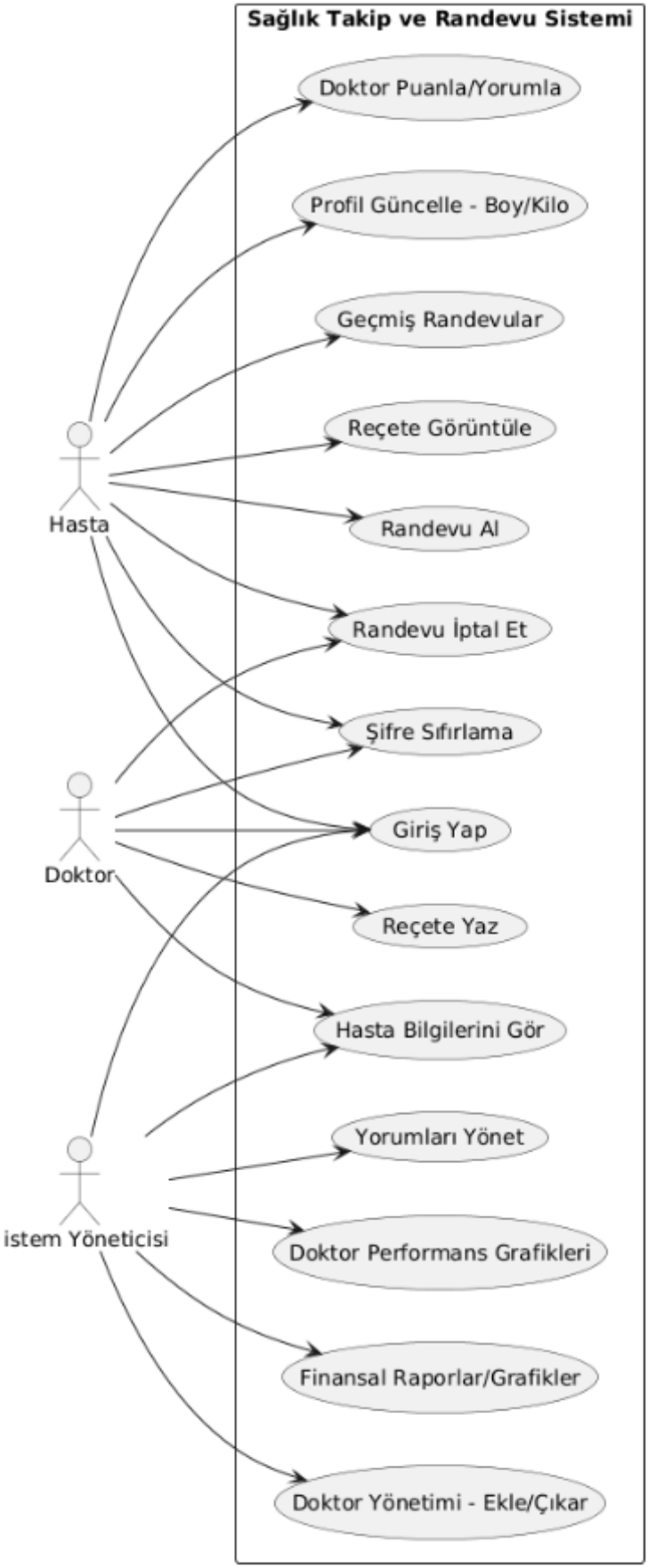
- **Ön Koşul:** Sistemde finansal verilerin (randevu ücretleri, doktor maaşları vb.) tanımlı olması.
- **Akış:** Admin "Finansal Raporlar" sekmesine girer; tarih aralığı seçer, sistem **Chart.js** veya benzeri bir kütüphaneyle aylık kar-zarar grafiği oluşturur.
- **Son Koşul:** Üst düzey yönetici için finansal özet sunulur.

Senaryo 10: Doktor Ekleme ve Çıkarma

- **Ön Koşul:** Admin yetkisi.
- **Akış:** Admin "Doktor Yönetimi" ne girer; yeni doktorun bilgilerini girip "Kaydet" der (Ekleme) veya aktif doktoru "Pasif" duruma getirir (Çıkarma).
- **Son Koşul:** Sistemin Users ve Doctors tabloları güncellenir.

Senaryo 11: Doktor Performans Grafiklerini Görüntüleme

- **Ön Koşul:** Doktorun aldığı randevu sayısı ve hasta yorumlarının birikmiş olması.
- **Akış:** Admin, doktor listesinden bir isim seçer; sistem o doktorun "Aylık Baktığı Hasta Sayısı" ve "Ortalama Yıldız Puanı" verilerini grafik olarak getirir.


Sağlık Takip ve Randevu Sistemi
Doktor Puanla/Yorumla
Profil Güncelle - Boy/Kilo
Geçmiş Randevular
Reçete Görüntüle
Randevu Al
Randevu İptal Et
Şifre Sıfırlama
Giriş Yap
Reçete Yaz
Hasta Bilgilerini Gör
Yorumları Yönet
Doktor Performans Grafikleri
Finansal Raporlar/Grafikler
Doktor Yönetimi - Ekle/Çıkar
Hasta
Doktor
İstem Yöneticisi

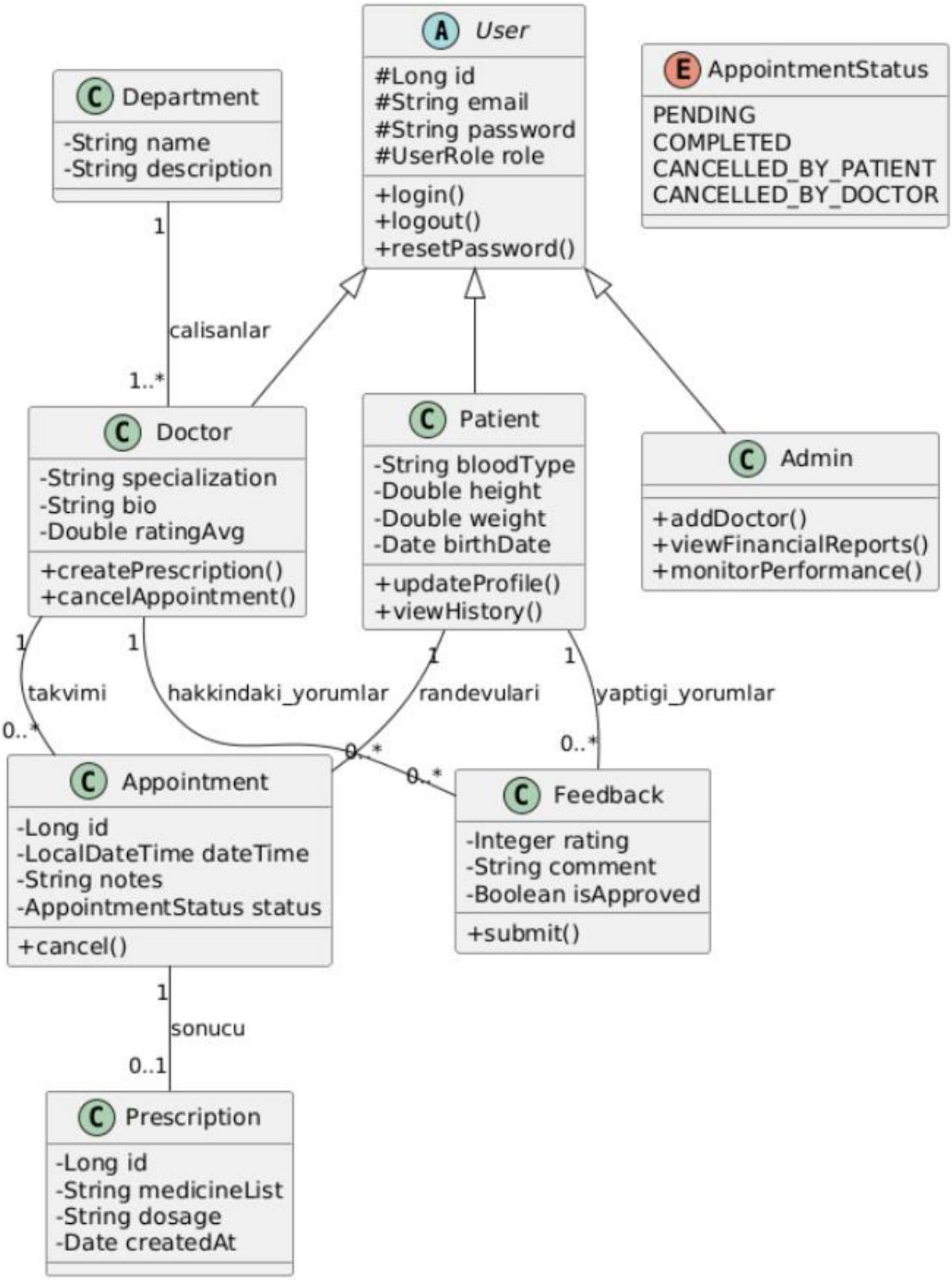
---

1. Sistem Sınıf Diyagramı (Class Diagram)

Bu diyagram, backend tarafında (Spring Boot) oluşturacağınız **Entity** sınıflarını ve aralarındaki ilişkileri temsil eder.

Sınıf Detayları ve Metotlar:

- **User (Abstract):** id, email, password, role. Metotlar: login(), logout(), resetPassword().
- **Patient (Extends User):** bloodType, height, weight, dob. Metotlar: bookAppointment(), viewPrescriptions(), giveFeedback().
- **Doctor (Extends User):** specialization, bio, ratingAvg. Metotlar: createPrescription(), updateSchedule(), cancelAppointment().
- **Admin (Extends User):** Metotlar: addDoctor(), viewFinancialReports(), analyzePerformance().
- **Appointment:** dateTime, status (Enum), notes.
- **Prescription:** medicineList, dosageInstructions, date.
- **Feedback:** rating (int), comment, isApproved.


User
#Long id
#String email
#String password
#UserRole role
+login()
+logout()
+resetPassword()
AppointmentStatus
PENDING
COMPLETED
CANCELLED_BY_PATIENT
CANCELLED_BY_DOCTOR
Department
-String name
-String description
1
calisanlar
1..*
Doctor
-String specialization
-String bio
-Double ratingAvg
+createPrescription()
+cancelAppointment()
Patient
-String bloodType
-Double height
-Double weight
-Date birthDate
+updateProfile()
+viewHistory()
Admin
+addDoctor()
+viewFinancialReports()
+monitorPerformance()
1
takvimi
0..*
1
hakkindaki_yorumlar
0..*
1
randevulari
0..*
1
yaptigi_yorumlar
0..*
Appointment
-Long id
-LocalDateTime dateTime
-String notes
-AppointmentStatus status
+cancel()
Feedback
-Integer rating
-String comment
-Boolean isApproved
+submit()
1
sonucu
0..1
Prescription
-Long id
-String medicineList
-String dosage
-Date createdAt

---

2. Sistem Ardışıklık Diyagramları (Sequence Diagrams)

Her bir senaryo için ana akışı (Happy Path) aşağıda kurguladım. Ata, raporun bu bölümünde "Aktör -> Arayüz -> Kontrolcü -> Veritabanı" akışını vurgulamalı.

A. Hasta Senaryoları İçin (Örn: Randevu Alma)

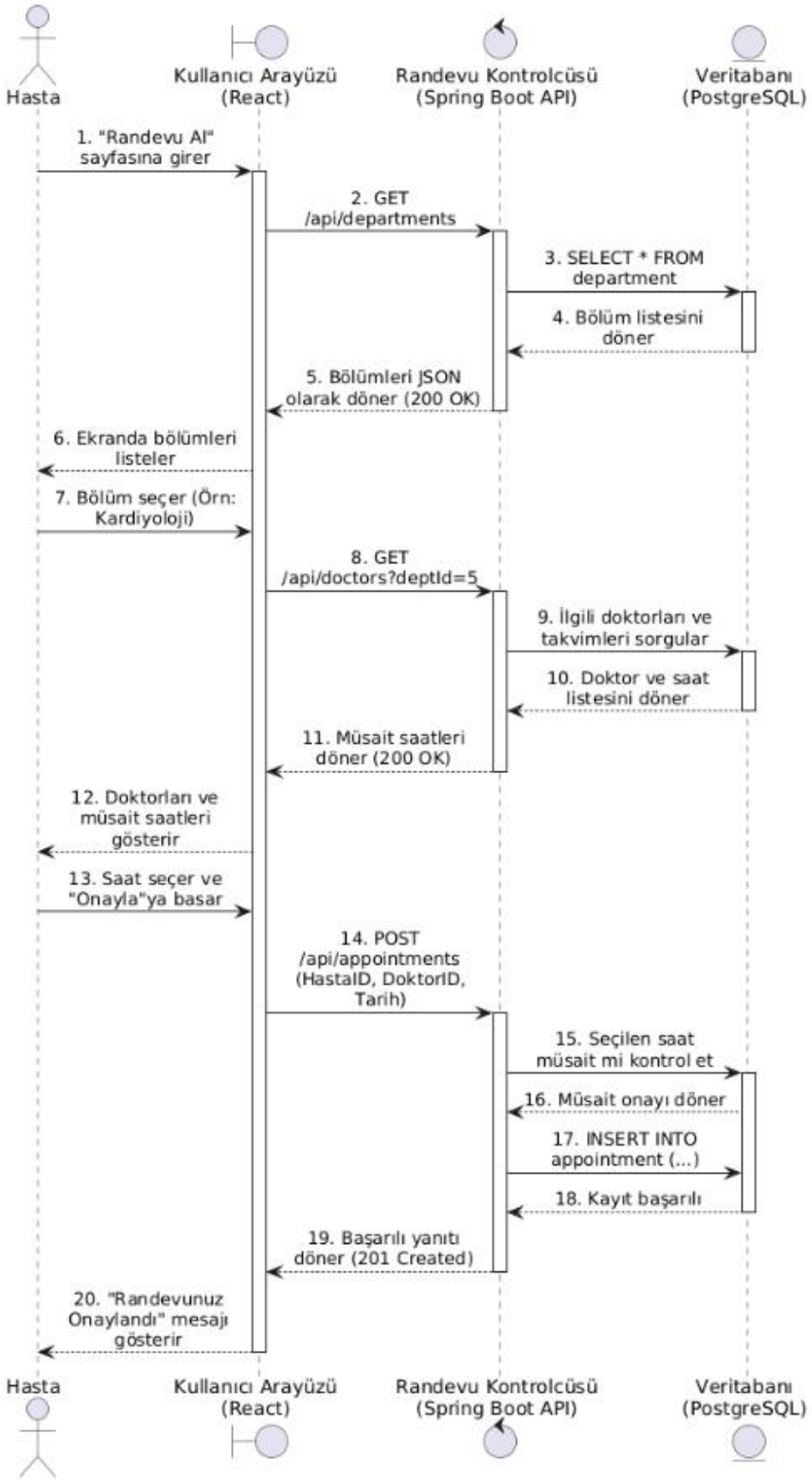
1. **Hasta**, arayüzden (React) poliklinik seçer.
2. **Arayüz**, **AppointmentController**'a müsait doktorları sorar.
3. **Controller**, **DoctorService** üzerinden Veri tabanından (PostgreSQL) listeyi çeker.
4. Hasta doktoru ve saati seçip onaylar.
5. **Controller**, randevuyu kaydeder ve başarılı mesajı döner.

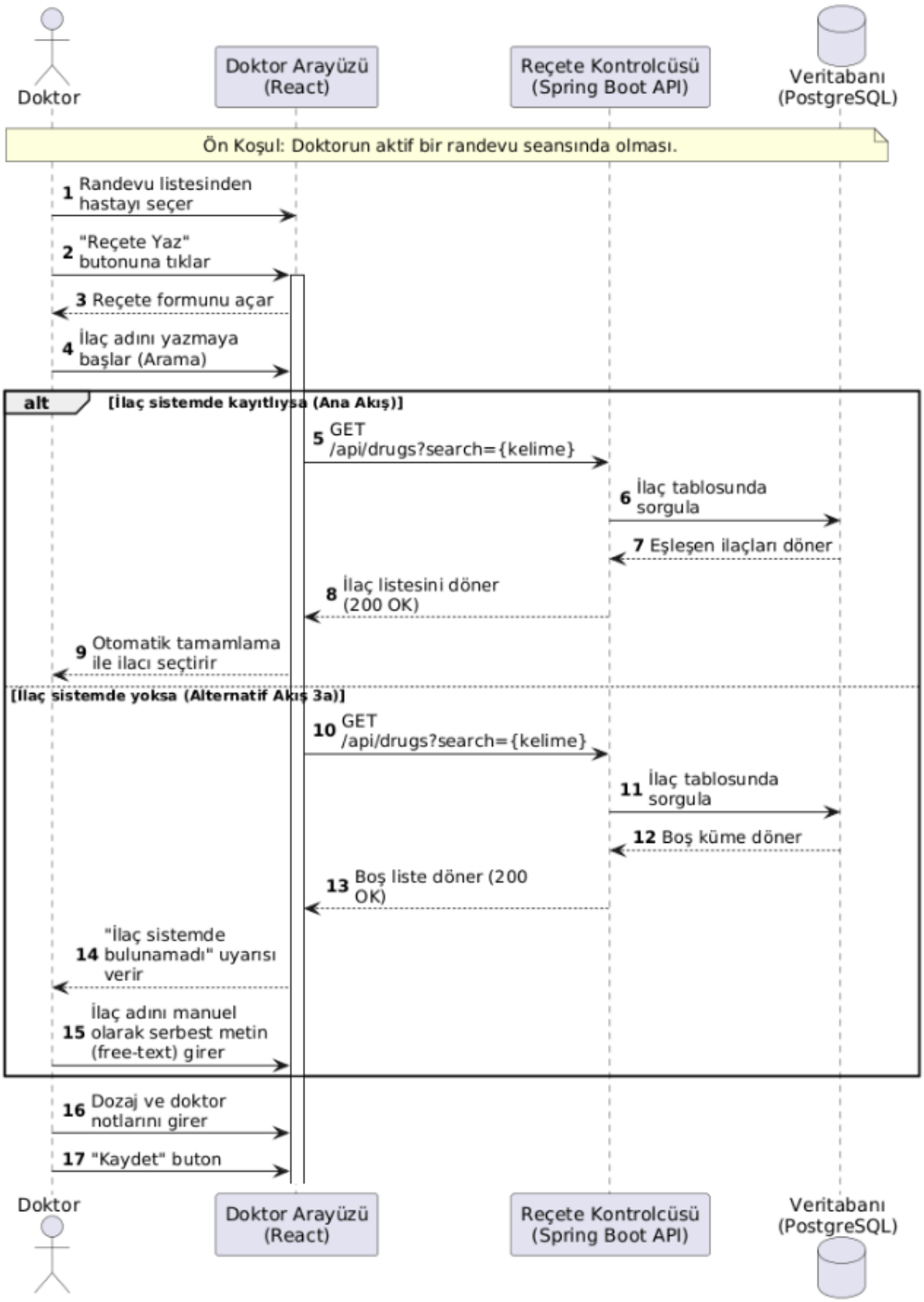
B. Doktor Senaryoları İçin (Örn: Reçete Oluşturma)

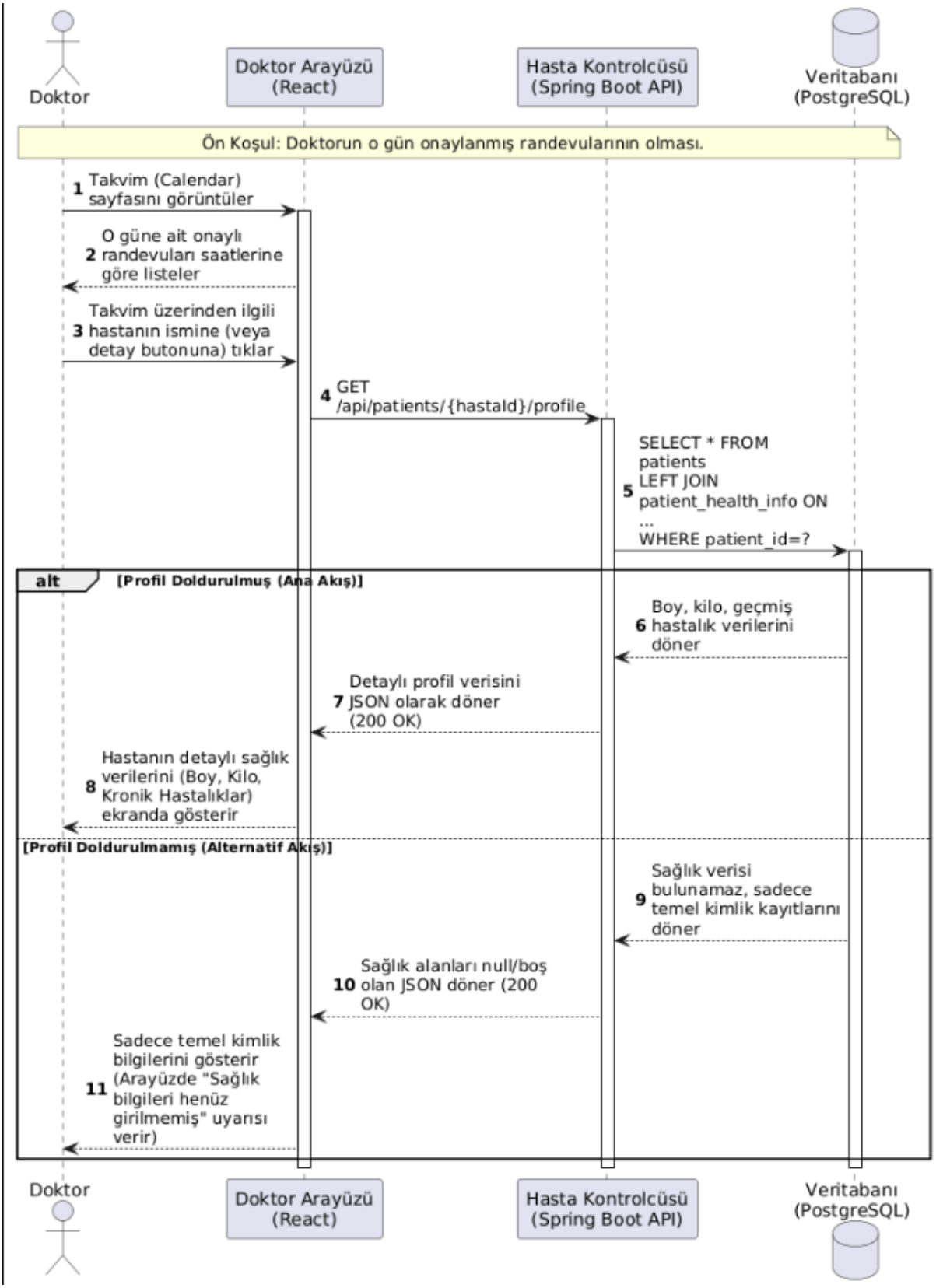
1. **Doktor**, aktif randevuyu seçer ve "Reçete Yaz" komutu gönderir.
2. **PrescriptionService**, girilen ilaç ve dozaj bilgilerini doğrular.
3. Veri tabanına yeni bir **Prescription** kaydı atılır ve **Appointment** durumu "Tamamlandı" olarak güncellenir.
4. Sistem, hastanın paneline "Yeni Reçete" bildirimi gönderir.

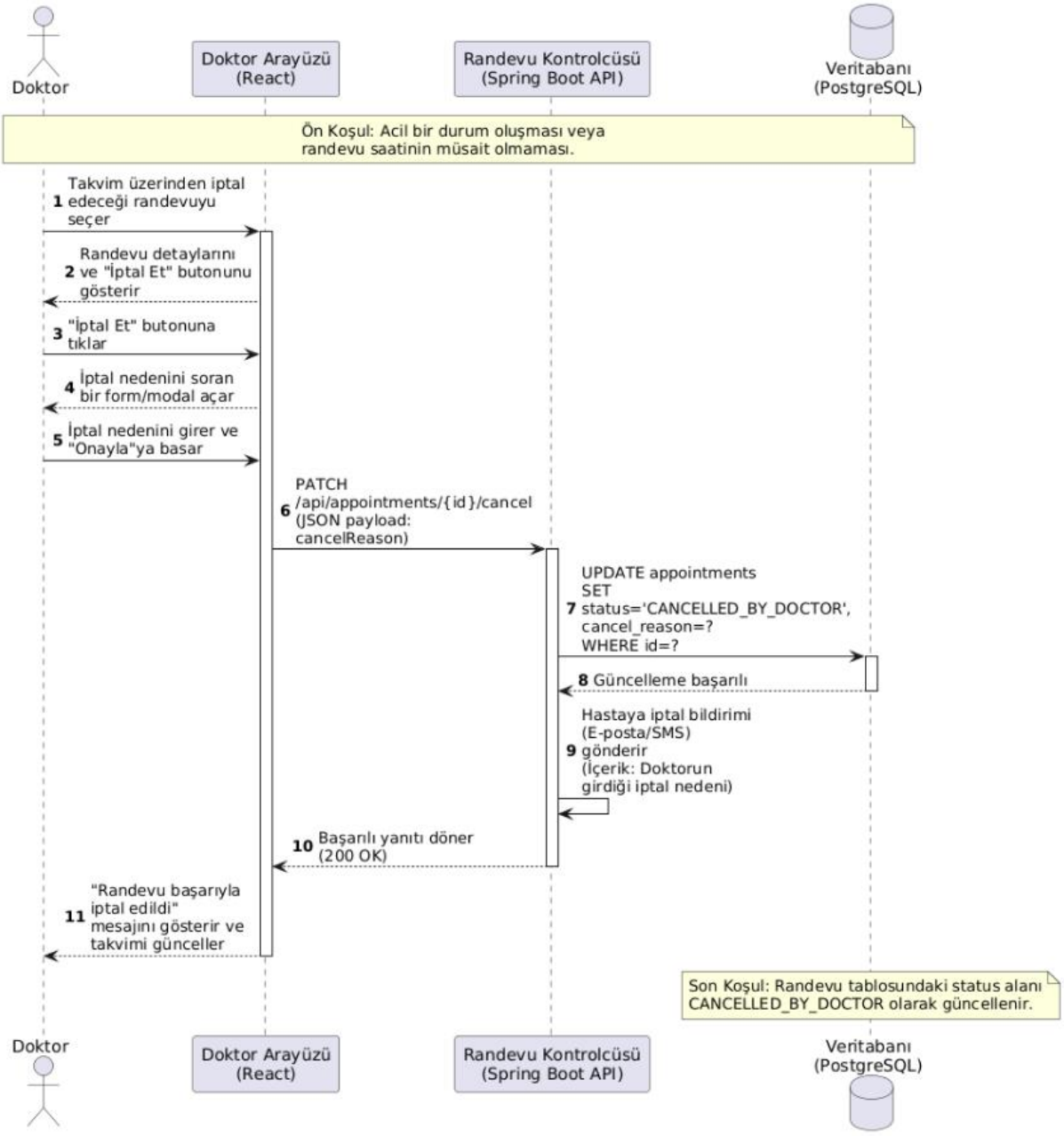
C. Admin Senaryoları İçin (Örn: Performans Grafiği Görüntüleme)

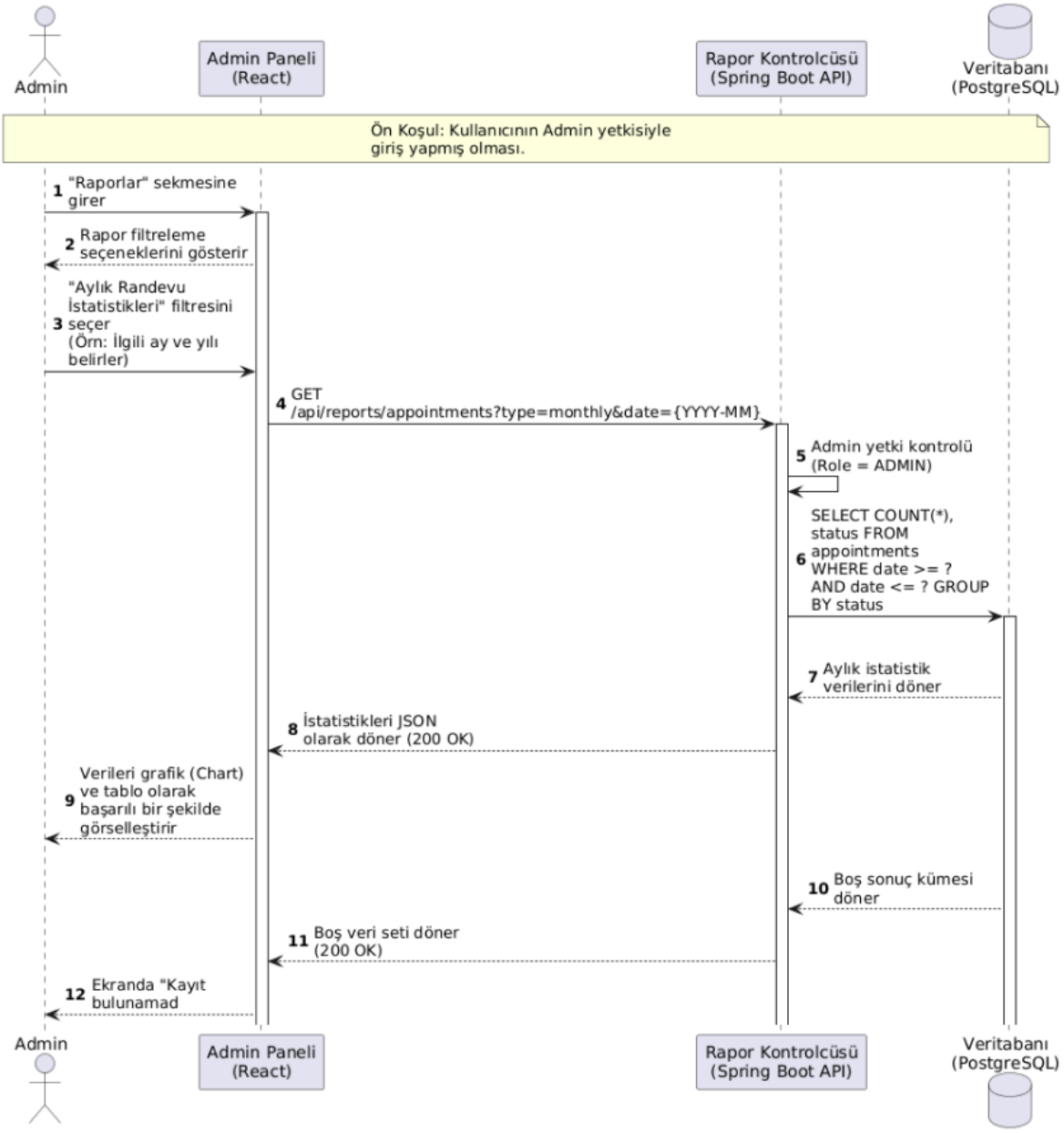
1. **Admin**, "İstatistikler" sayfasına girer.
2. **ReportController**, **AppointmentRepository**'den doktor bazlı randevu sayılarını ve **FeedbackRepository**'den ortalama puanları çeker.
3. Backend, bu verileri JSON formatında (grafiğe uygun) düzenleyip arayüze yollar.
4. **Arayüz (React-Chartjs)**, bu verileri görsel bir grafiğe dönüştürür.

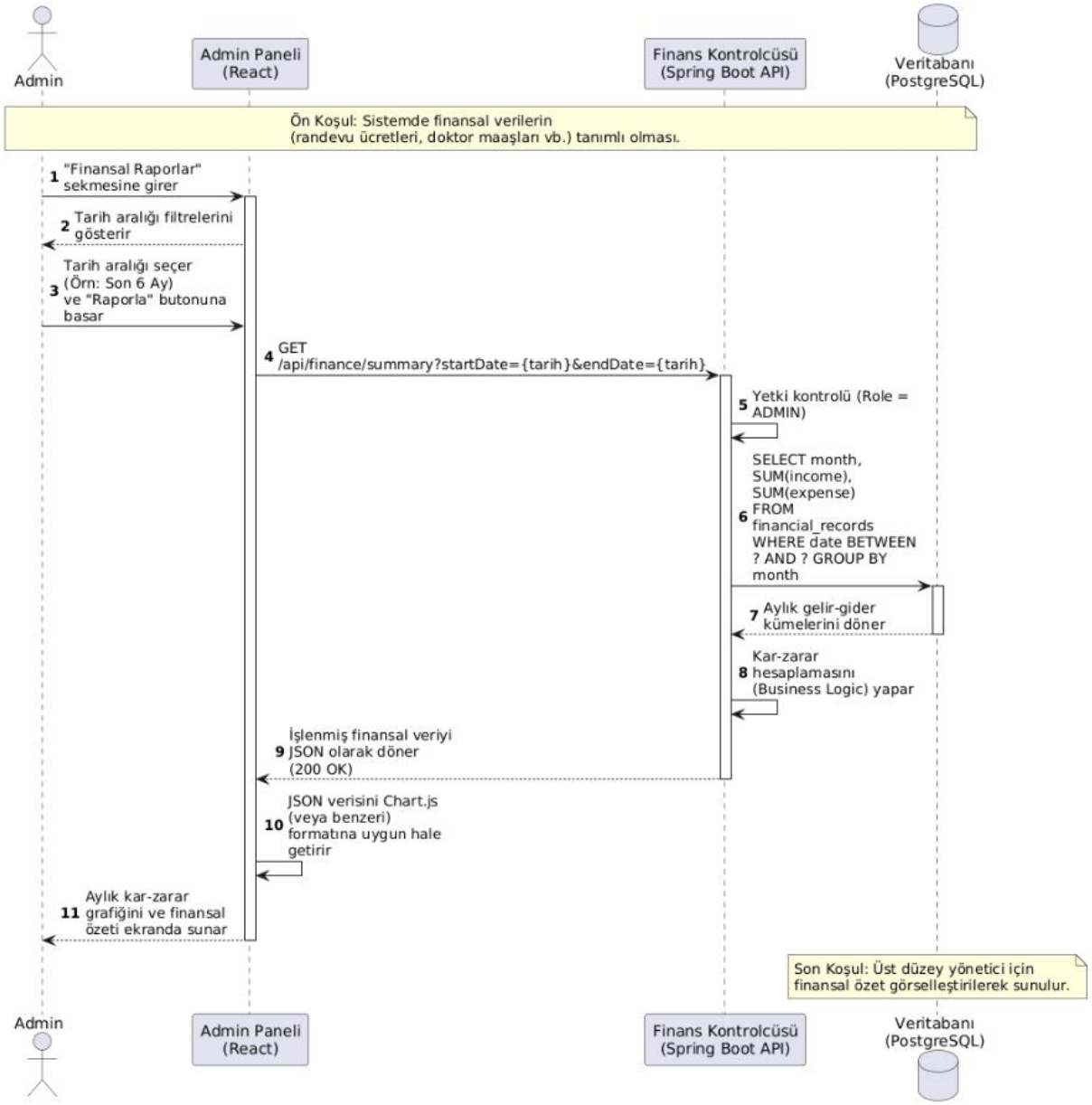

Hasta
Kullanıcı Arayüzü (React)
Randevu Kontrolcüsü (Spring Boot API)
Veritabanı (PostgreSQL)
1. "Randevu Al" sayfasına girer
2. GET /api/departments
3. SELECT * FROM department
4. Bölüm listesini döner
5. Bölümleri JSON olarak döner (200 OK)
6. Ekranda bölümleri listeler
7. Bölüm seçer (Örn: Kardiyoloji)
8. GET /api/doctors?deptId=5
9. İlgili doktorları ve takvimleri sorgular
10. Doktor ve saat listesini döner
11. Müsait saatleri döner (200 OK)
12. Doktorları ve müsait saatleri gösterir
13. Saat seçer ve "Onayla"ya basar
14. POST /api/appointments (HastaID, DoktorID, Tarih)
15. Seçilen saat müsait mi kontrol et
16. Müsait onayı döner
17. INSERT INTO appointment (...)
18. Kayıt başarılı
19. Başarılı yanıtı döner (201 Created)
20. "Randevunuz Onaylandı" mesajı gösterir
Hasta
Kullanıcı Arayüzü (React)
Randevu Kontrolcüsü (Spring Boot API)
Veritabanı (PostgreSQL)

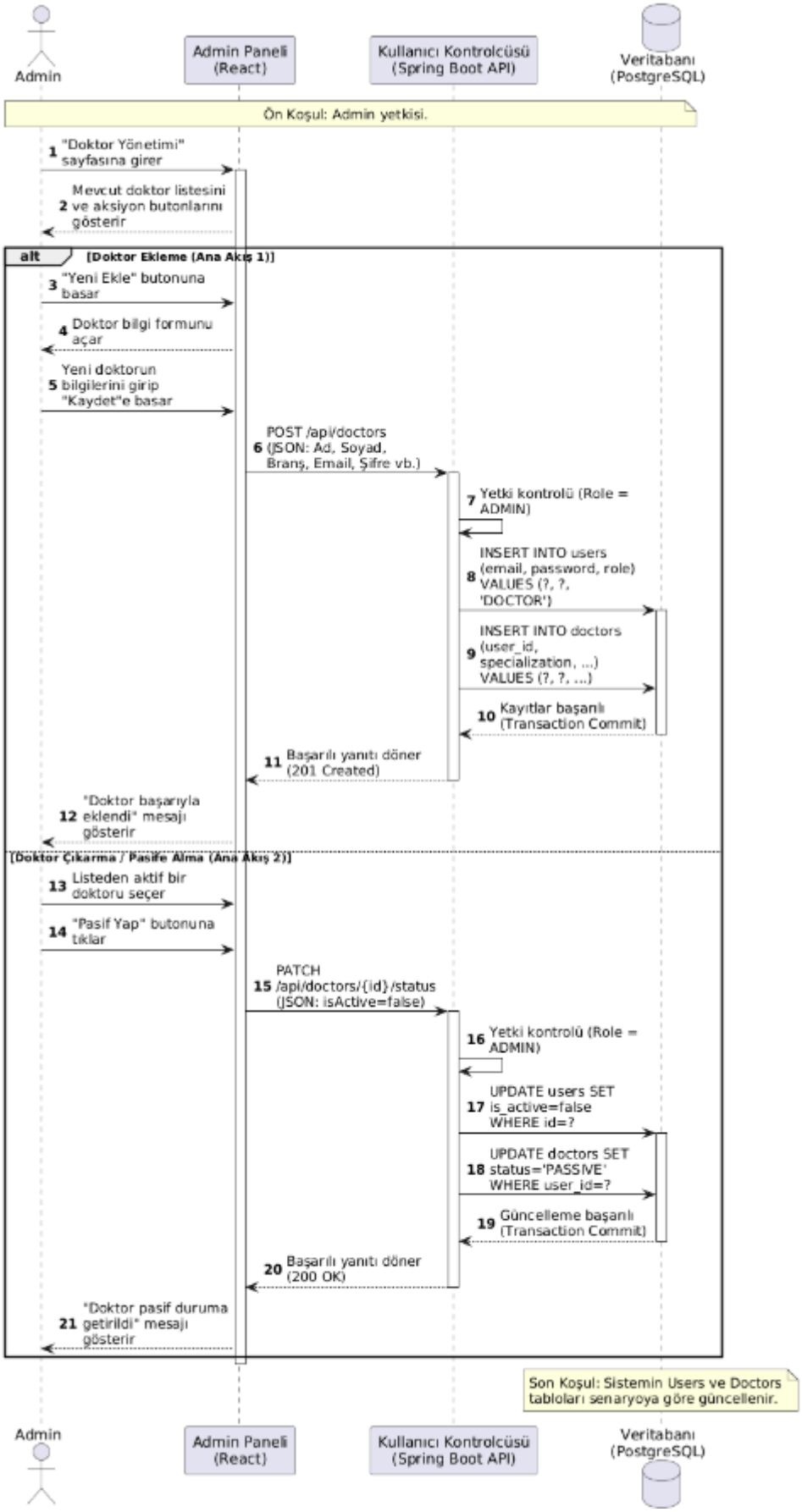

Doktor
Doktor Arayüzü (React)
Reçete Kontrolcüsü (Spring Boot API)
Veritabanı (PostgreSQL)
Ön Koşul: Doktorun aktif bir randevu seansında olması.
1 Randevu listesinden hastayı seçer
2 "Reçete Yaz" butonuna tıklar
3 Reçete formunu açar
4 İlaç adını yazmaya başlar (Arama)
alt
[İlaç sistemde kayıtlıysa (Ana Akış)]
5 GET /api/drugs?search={kelime}
6 İlaç tablosunda sorgula
7 Eşleşen ilaçları döner
8 İlaç listesini döner (200 OK)
9 Otomatik tamamlama ile ilacı seçtirir
[İlaç sistemde yoksa (Alternatif Akış 3a)]
10 GET /api/drugs?search={kelime}
11 İlaç tablosunda sorgula
12 Boş küme döner
13 Boş liste döner (200 OK)
14 "İlaç sistemde bulunamadı" uyarısı verir
15 İlaç adını manuel olarak serbest metin (free-text) girer
16 Dozaj ve doktor notlarını girer
17 "Kaydet" buton
Doktor
Doktor Arayüzü (React)
Reçete Kontrolcüsü (Spring Boot API)
Veritabanı (PostgreSQL)

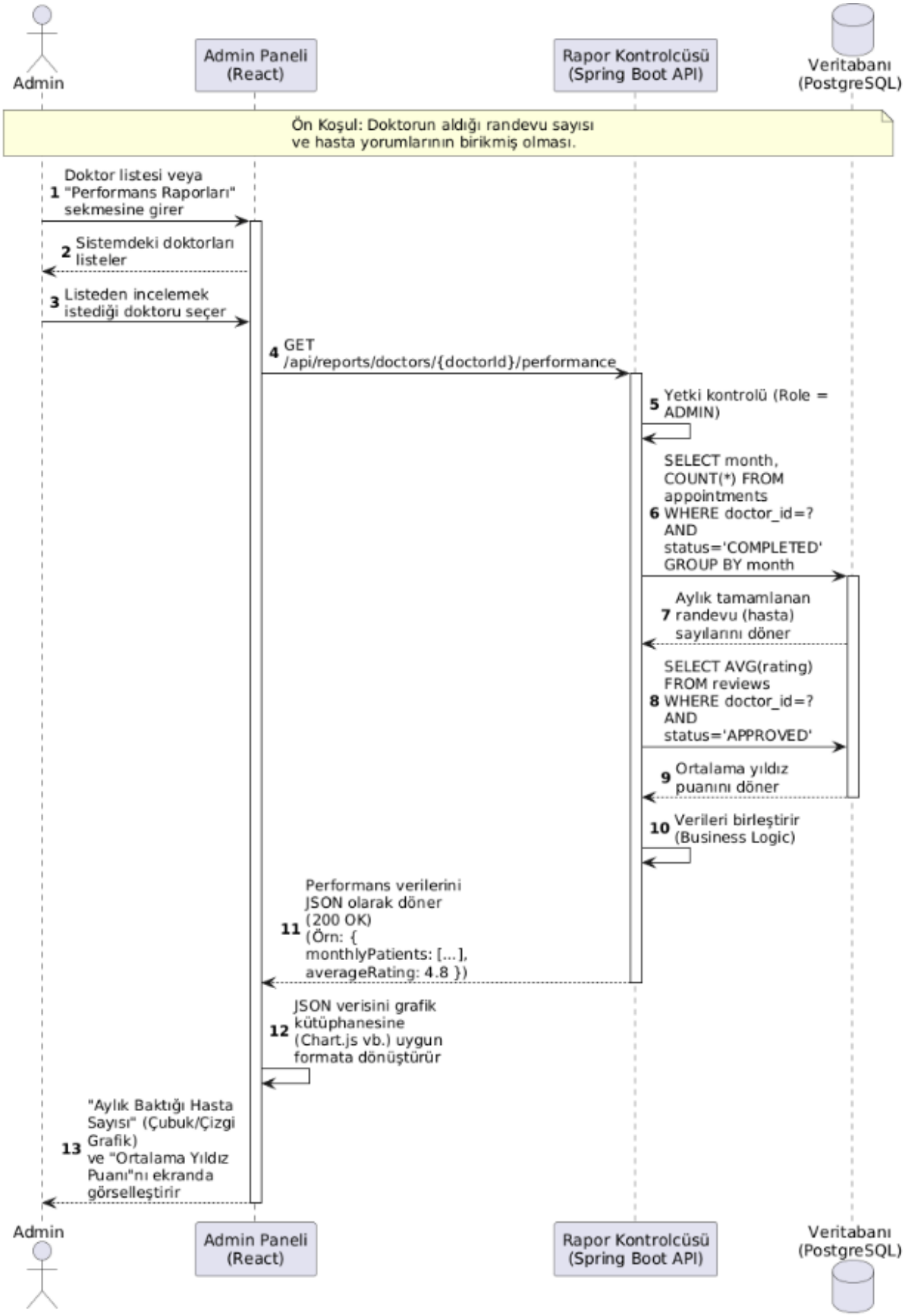

Doktor
Doktor Arayüzü (React)
Hasta Kontrolcüsü (Spring Boot API)
Veritabanı (PostgreSQL)
Ön Koşul: Doktorun o gün onaylanmış randevularının olması.
1 Takvim (Calendar) sayfasını görüntüler
2 O güne ait onaylı randevuları saatlerine göre listeler
3 Takvim üzerinden ilgili hastanın ismine (veya detay butonuna) tıklar
4 GET /api/patients/{hastaId}/profile
5 SELECT * FROM patients LEFT JOIN patient_health_info ON ... WHERE patient_id=?
alt
[Profil Doldurulmuş (Ana Akış)]
6 Boy, kilo, geçmiş hastalık verilerini döner
7 Detaylı profil verisini JSON olarak döner (200 OK)
8 Hastanın detaylı sağlık verilerini (Boy, Kilo, Kronik Hastalıklar) ekranda gösterir
[Profil Doldurulmamış (Alternatif Akış)]
9 Sağlık verisi bulunamaz, sadece temel kimlik kayıtlarını döner
10 Sağlık alanları null/boş olan JSON döner (200 OK)
11 Sadece temel kimlik bilgilerini gösterir (Arayüzde "Sağlık bilgileri henüz girilmemiş" uyarısı verir)
Doktor
Doktor Arayüzü (React)
Hasta Kontrolcüsü (Spring Boot API)
Veritabanı (PostgreSQL)

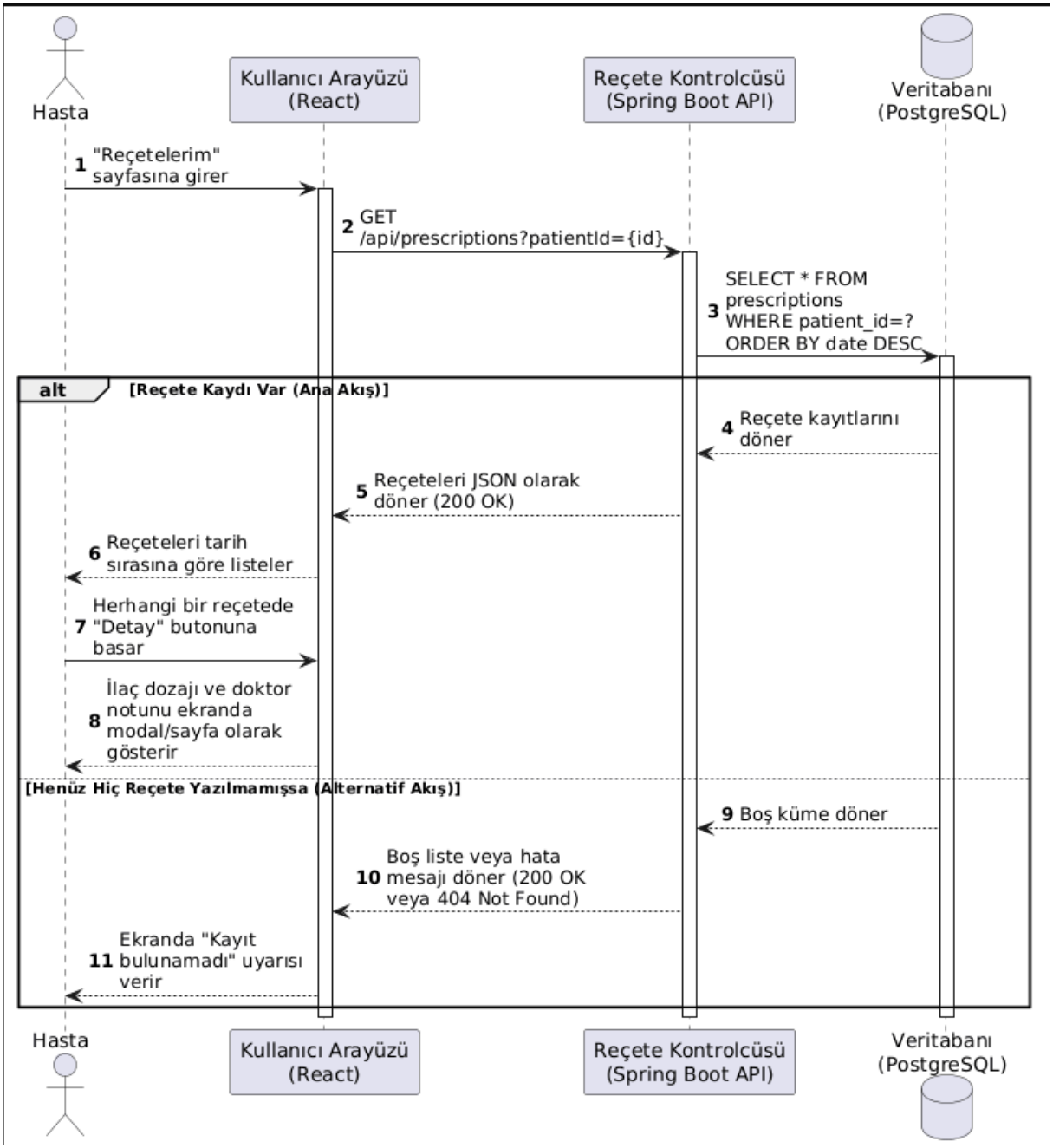

Doktor
Doktor Arayüzü (React)
Randevu Kontrolcüsü (Spring Boot API)
Veritabanı (PostgreSQL)
Ön Koşul: Acil bir durum oluşması veya randevu saatinin müsait olmaması.
1 Takvim üzerinden iptal edeceği randevuyu seçer
2 Randevu detaylarını ve "İptal Et" butonunu gösterir
3 "İptal Et" butonuna tıklar
4 İptal nedenini soran bir form/modal açar
5 İptal nedenini girer ve "Onayla"ya basar
6 PATCH /api/appointments/{id}/cancel (JSON payload: cancelReason)
7 UPDATE appointments SET status='CANCELLED_BY_DOCTOR', cancel_reason=? WHERE id=?
8 Güncelleme başarılı
9 Hastaya iptal bildirimi (E-posta/SMS) gönderir (İçerik: Doktorun girdiği iptal nedeni)
10 Başarılı yanıtı döner (200 OK)
11 "Randevu başarıyla iptal edildi" mesajını gösterir ve takvimi günceller
Son Koşul: Randevu tablosundaki status alanı CANCELLED_BY_DOCTOR olarak güncellenir.
Doktor
Doktor Arayüzü (React)
Randevu Kontrolcüsü (Spring Boot API)
Veritabanı (PostgreSQL)

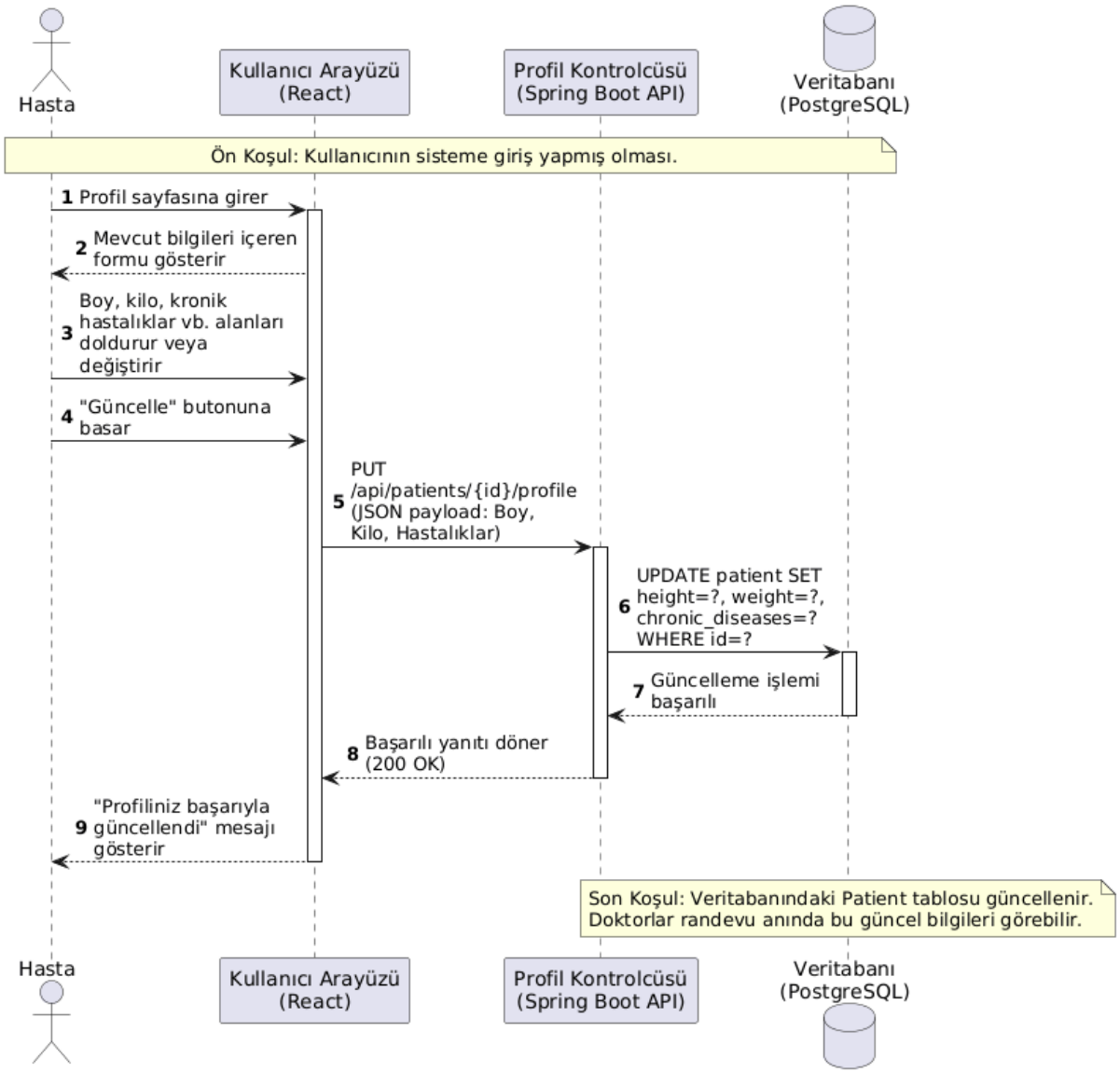

Admin
Admin Paneli (React)
Rapor Kontrolcüsü (Spring Boot API)
Veritabanı (PostgreSQL)
Ön Koşul: Kullanıcının Admin yetkisiyle giriş yapmış olması.
1 "Raporlar" sekmesine girer
2 Rapor filtreleme seçeneklerini gösterir
3 "Aylık Randevu İstatistikleri" filtresini seçer (Örn: İlgili ay ve yılı belirler)
4 GET /api/reports/appointments?type=monthly&date={YYYY-MM}
5 Admin yetki kontrolü (Role = ADMIN)
6 SELECT COUNT(*), status FROM appointments WHERE date >= ? AND date <= ? GROUP BY status
7 Aylık istatistik verilerini döner
8 İstatistikleri JSON olarak döner (200 OK)
9 Verileri grafik (Chart) ve tablo olarak başarılı bir şekilde görselleştirir
10 Boş sonuç kümesi döner
11 Boş veri seti döner (200 OK)
12 Ekranda "Kayıt bulunamad
Admin
Admin Paneli (React)
Rapor Kontrolcüsü (Spring Boot API)
Veritabanı (PostgreSQL)

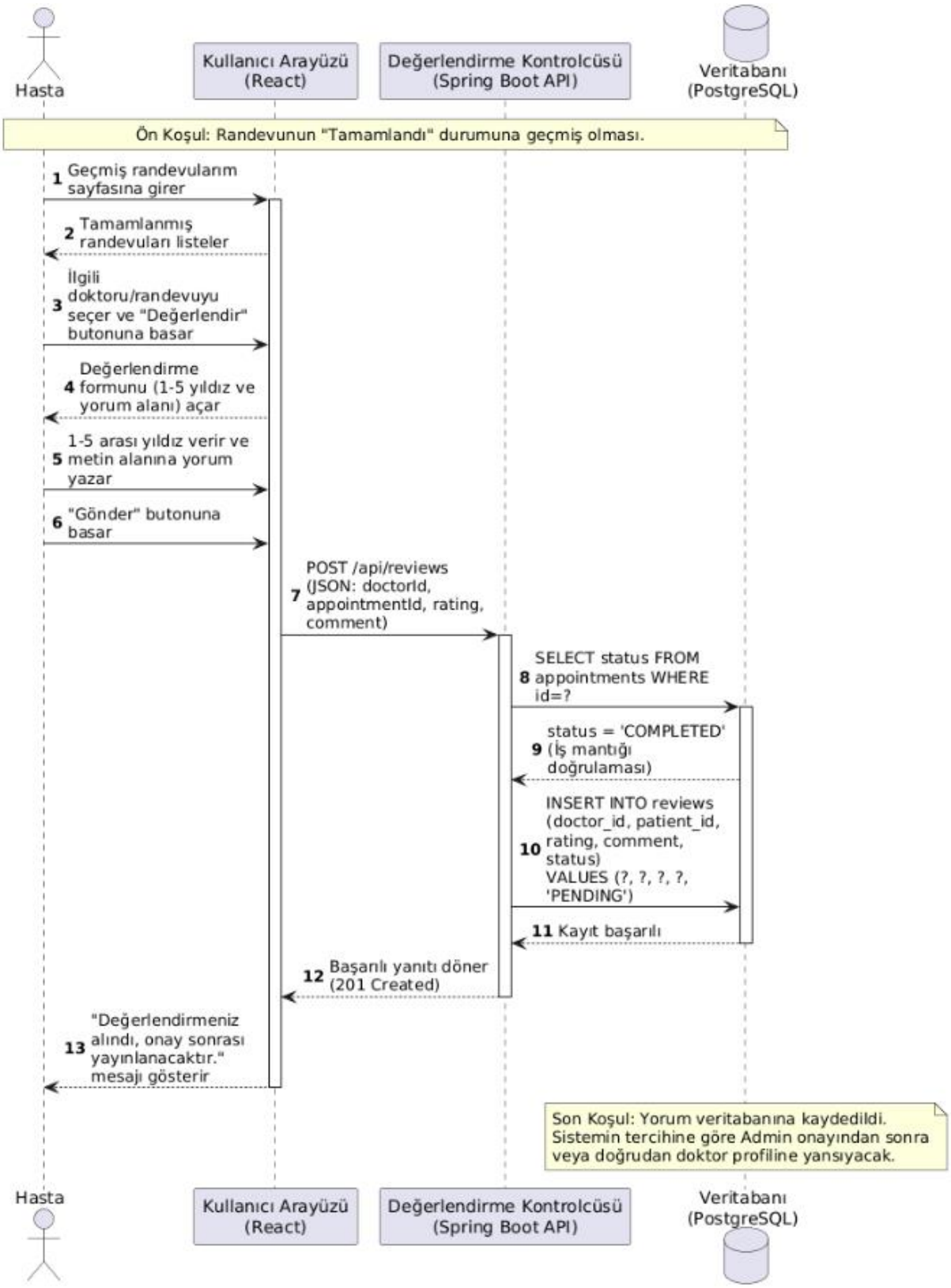

Admin
Admin Paneli (React)
Finans Kontrolcüsü (Spring Boot API)
Veritabanı (PostgreSQL)
Ön Koşul: Sistemde finansal verilerin (randevu ücretleri, doktor maaşları vb.) tanımlı olması.
1 "Finansal Raporlar" sekmesine girer
2 Tarih aralığı filtrelerini gösterir
3 Tarih aralığı seçer (Örn: Son 6 Ay) ve "Raporla" butonuna basar
4 GET /api/finance/summary?startDate={tarih}&endDate={tarih}
5 Yetki kontrolü (Role = ADMIN)
6 SELECT month, SUM(income), SUM(expense) FROM financial_records WHERE date BETWEEN ? AND ? GROUP BY month
7 Aylık gelir-gider kümelerini döner
8 Kar-zarar hesaplamasını (Business Logic) yapar
9 İşlenmiş finansal veriyi JSON olarak döner (200 OK)
10 JSON verisini Chart.js (veya benzeri) formatına uygun hale getirir
11 Aylık kar-zarar grafiğini ve finansal özeti ekranda sunar
Son Koşul: Üst düzey yönetici için finansal özet görselleştirilerek sunulur.
Admin
Admin Paneli (React)
Finans Kontrolcüsü (Spring Boot API)
Veritabanı (PostgreSQL)


Admin
Admin Paneli (React)
Kullanıcı Kontrolcüsü (Spring Boot API)
Veritabanı (PostgreSQL)
Ön Koşul: Admin yetkisi.
1 "Doktor Yönetimi" sayfasına girer
2 Mevcut doktor listesini ve aksiyon butonlarını gösterir
alt [Doktor Ekleme (Ana Akış 1)]
3 "Yeni Ekle" butonuna basar
4 Doktor bilgi formunu açar
5 Yeni doktorun bilgilerini girip "Kaydet"e basar
6 POST /api/doctors (JSON: Ad, Soyad, Branş, Email, Şifre vb.)
7 Yetki kontrolü (Role = ADMIN)
8 INSERT INTO users (email, password, role) VALUES (?, ?, 'DOCTOR')
9 INSERT INTO doctors (user_id, specialization, ...) VALUES (?, ?, ...)
10 Kayıtlar başarılı (Transaction Commit)
11 Başarılı yanıtı döner (201 Created)
12 "Doktor başarıyla eklendi" mesajı gösterir
[Doktor Çıkarma / Pasife Alma (Ana Akış 2)]
13 Listeden aktif bir doktoru seçer
14 "Pasif Yap" butonuna tıklar
15 PATCH /api/doctors/{id}/status (JSON: isActive=false)
16 Yetki kontrolü (Role = ADMIN)
17 UPDATE users SET is_active=false WHERE id=?
18 UPDATE doctors SET status='PASSIVE' WHERE user_id=?
19 Güncelleme başarılı (Transaction Commit)
20 Başarılı yanıtı döner (200 OK)
21 "Doktor pasif duruma getirildi" mesajı gösterir
Son Koşul: Sistemin Users ve Doctors tabloları senaryoya göre güncellenir.
Admin
Admin Paneli (React)
Kullanıcı Kontrolcüsü (Spring Boot API)
Veritabanı (PostgreSQL)


Admin
Admin Paneli (React)
Rapor Kontrolcüsü (Spring Boot API)
Veritabanı (PostgreSQL)
Ön Koşul: Doktorun aldığı randevu sayısı ve hasta yorumlarının birikmiş olması.
1 Doktor listesi veya "Performans Raporları" sekmesine girer
2 Sistemdeki doktorları listeler
3 Listeden incelemek istediği doktoru seçer
4 GET /api/reports/doctors/{doctorId}/performance
5 Yetki kontrolü (Role = ADMIN)
6 SELECT month, COUNT(*) FROM appointments WHERE doctor_id=? AND status='COMPLETED' GROUP BY month
7 Aylık tamamlanan randevu (hasta) sayılarını döner
8 SELECT AVG(rating) FROM reviews WHERE doctor_id=? AND status='APPROVED'
9 Ortalama yıldız puanını döner
10 Verileri birleştirir (Business Logic)
11 Performans verilerini JSON olarak döner (200 OK) (Örn: { monthlyPatients: [...], averageRating: 4.8 })
12 JSON verisini grafik kütüphanesine (Chart.js vb.) uygun formata dönüştürür
13 "Aylık Baktığı Hasta Sayısı" (Çubuk/Çizgi Grafik) ve "Ortalama Yıldız Puanı"nı ekranda görselleştirir
Admin
Admin Paneli (React)
Rapor Kontrolcüsü (Spring Boot API)
Veritabanı (PostgreSQL)


Hasta
Kullanıcı Arayüzü (React)
Reçete Kontrolcüsü (Spring Boot API)
Veritabanı (PostgreSQL)
1 "Reçetelerim" sayfasına girer
2 GET /api/prescriptions?patientId={id}
3 SELECT * FROM prescriptions WHERE patient_id=? ORDER BY date DESC
alt
[Reçete Kaydı Var (Ana Akış)]
4 Reçete kayıtlarını döner
5 Reçeteleri JSON olarak döner (200 OK)
6 Reçeteleri tarih sırasına göre listeler
7 Herhangi bir reçetede "Detay" butonuna basar
8 İlaç dozajı ve doktor notunu ekranda modal/sayfa olarak gösterir
[Henüz Hiç Reçete Yazılmamışsa (Alternatif Akış)]
9 Boş küme döner
10 Boş liste veya hata mesajı döner (200 OK veya 404 Not Found)
11 Ekranda "Kayıt bulunamadı" uyarısı verir
Hasta
Kullanıcı Arayüzü (React)
Reçete Kontrolcüsü (Spring Boot API)
Veritabanı (PostgreSQL)


Hasta
Kullanıcı Arayüzü (React)
Profil Kontrolcüsü (Spring Boot API)
Veritabanı (PostgreSQL)
Ön Koşul: Kullanıcının sisteme giriş yapmış olması.
1 Profil sayfasına girer
2 Mevcut bilgileri içeren formu gösterir
3 Boy, kilo, kronik hastalıklar vb. alanları doldurur veya değiştirir
4 "Güncelle" butonuna basar
5 PUT /api/patients/{id}/profile (JSON payload: Boy, Kilo, Hastalıklar)
6 UPDATE patient SET height=?, weight=?, chronic_diseases=? WHERE id=?
7 Güncelleme işlemi başarılı
8 Başarılı yanıtı döner (200 OK)
9 "Profiliniz başarıyla güncellendi" mesajı gösterir
Son Koşul: Veritabanındaki Patient tablosu güncellenir. Doktorlar randevu anında bu güncel bilgileri görebilir.
Hasta
Kullanıcı Arayüzü (React)
Profil Kontrolcüsü (Spring Boot API)
Veritabanı (PostgreSQL)


Hasta
Kullanıcı Arayüzü (React)
Değerlendirme Kontrolcüsü (Spring Boot API)
Veritabanı (PostgreSQL)
Ön Koşul: Randevunun "Tamamlandı" durumuna geçmiş olması.
1 Geçmiş randevularım sayfasına girer
2 Tamamlanmış randevuları listeler
3 İlgili doktoru/randevuyu seçer ve "Değerlendir" butonuna basar
4 Değerlendirme formunu (1-5 yıldız ve yorum alanı) açar
5 1-5 arası yıldız verir ve metin alanına yorum yazar
6 "Gönder" butonuna basar
7 POST /api/reviews (JSON: doctorId, appointmentId, rating, comment)
8 SELECT status FROM appointments WHERE id=?
9 status = 'COMPLETED' (İş mantığı doğrulaması)
10 INSERT INTO reviews (doctor_id, patient_id, rating, comment, status) VALUES (?, ?, ?, ?, 'PENDING')
11 Kayıt başarılı
12 Başarılı yanıtı döner (201 Created)
13 "Değerlendirmeniz alındı, onay sonrası yayınlanacaktır." mesajı gösterir
Son Koşul: Yorum veritabanına kaydedildi. Sistemin tercihine göre Admin onayından sonra veya doğrudan doktor profiline yansıyacak.
Hasta
Kullanıcı Arayüzü (React)
Değerlendirme Kontrolcüsü (Spring Boot API)
Veritabanı (PostgreSQL)